

```
In[1]:= ClearAll["Global`*"];
$HistoryLength = 0;
```

```
metadata = <|"Version" → $Version, "System" → $System,
  "Processor" → $ProcessorType, "Date" → DateString["ISODateTime"] |>;
```

```
SetDirectory[NotebookDirectory[]];
metadata
```

```
Out[5]= <|Version → 15.0.1 for Mac OS X ARM (64-bit) (July 2, 2026),
  System → Mac OS X ARM (64-bit), Processor → ARM64, Date → 2026-08-17T16:43:24 |>
```

```
In[6]:= ClearAll[alpha, t, w, delta];
```

```
vars = {alpha, t, w, delta};
```

```
f = {1 / alpha + w / t^2 + delta - 2,
  w / t^3 + delta - 1, 3 w / t^4 + 3 delta - 2, 4 w / t^5 + 4 delta - 2};
```

```
positiveSolution =
  FullSimplify[Reduce[And @@ Thread[f == 0] && And @@ (# > 0 & /@ vars), vars, Reals]];
```

```
jac = Table[D[f[[i]], vars[[j]]], {i, 4}, {j, 4}];
point = Thread[vars → {3, 2, 16 / 3, 1 / 3}];
jacAtPoint = jac /. point;
jacInverse = Inverse[jacAtPoint];
```

```
m3Results = <|"PositiveSolution" → positiveSolution,
  "Jacobian" → jacAtPoint, "Determinant" → Det[jacAtPoint],
  "Inverse" → jacInverse, "InfinityNorm" → Max[Total[Abs[#]] & /@ jacInverse] |>;
```

```
m3Results // InputForm
```

```
Out[15]//InputForm=
<|"PositiveSolution" -> alpha == 3 && t == 2 && 3*w == 16 && 3*delta == 1,
  "Jacobian" -> {{-1/9, -4/3, 1/4, 1}, {0, -1, 1/8, 1}, {0, -2, 3/16, 3},
    {0, -5/3, 1/8, 4}}, "Determinant" -> -1/144,
  "Inverse" -> {{-9, 45, -24, 9}, {0, 6, -6, 3}, {0, 48, -112/3, 16}, {0, 1, -4/3, 1},
  "InfinityNorm" -> 304/3 |>
```

```
In[16]:= ClearAll[k, m, d, aa, bb, cc, ddp, lam, coef];
```

```
(*Intended domain:d,m,k are integers,0≤m≤d,0≤k≤d-m;
aa,bb,cc,ddp are positive.*)
```

```
coef[k_] := Pochhammer[m - d, k] * Pochhammer[aa + m, k] * Pochhammer[cc + m, k] *
  lam^k / (Pochhammer[bb + m, k] * Pochhammer[ddp + m, k] * k!);
```

```
stepRules = {HoldPattern[Pochhammer[x_, 1 + k]] => (x + k) Pochhammer[x, k],
  HoldPattern[Factorial[1 + k]] => (1 + k) Factorial[k],
  HoldPattern[lam^ (1 + k)] => lam lam^k};
```

```
ratioRaw = coef[k + 1] / coef[k];
```

```
ratioFromSteps = Factor@Cancel[ratioRaw /. stepRules];
```

```
coefficientRatio = Factor[ratioFromSteps /. lam -> ddp / (aa cc)];
```

```
coefficientRatio // InputForm
```

```
In[23]:= (ddp*(aa + k + m)*(cc + k + m)*(-d + k + m))/(aa*cc*(1 + k)*(bb + k + m)*(ddp + k +
Out[23]:=

$$\frac{\text{ddp} (aa + k + m) (cc + k + m) (-d + k + m)}{aa \text{ cc} (1 + k) (bb + k + m) (ddp + k + m)}$$

```

```
In[24]:= crossMultipliedCheck =
  Factor@Cancel[(k + 1) * (k + bb + m) * (k + ddp + m) * coef[k + 1] - lam *
    (k + m - d) * (k + aa + m) * (k + cc + m) * coef[k]) /. stepRules];
```

```
crossMultipliedCheck // InputForm
```

In[26]:= 0

```

ClearAll[
  y, g, theta, shift, lhs, rhs, ode,
  lambda0, odeDerivativeCoefficients
];

lambda0 = ddp/(aa cc);

theta[expr_] := Expand[y D[expr, y]];

shift[expr_, c0_] :=
  Expand[theta[expr] + c0 expr];

lhs =
  theta[
    shift[
      shift[g[y], ddp + m - 1],
      bb + m - 1
    ]
  ];

rhs =
  lambda0 y *
  shift[
    shift[
      shift[g[y], cc + m],
      aa + m
    ],
    m - d
  ];

ode = Expand[lhs - rhs];

odeDerivativeCoefficients = {
  Factor[Coefficient[ode, g[y]]],
  Factor[Coefficient[ode, g'[y]]],
  Factor[Coefficient[ode, g''[y]]],
  Factor[Coefficient[ode, g'''[y]]]
};

odeDerivativeCoefficients // InputForm

```

Out[26]=

0

Out[44]//InputForm=

```

{ (ddp*(d - m)*(aa + m)*(cc + m)*y)/(aa*cc),
  (y*(aa*bb*cc*ddp + aa*bb*cc*m + aa*cc*ddp*m + aa*cc*m^2 - ddp*y - aa*ddp*y - cc*ddp*
    aa*d*ddp*y + cc*d*ddp*y - 3*ddp*m*y - 2*aa*ddp*m*y - 2*cc*ddp*m*y + 2*d*ddp*m*y
    - (y^2*(-(aa*cc) - aa*bb*cc - aa*cc*ddp - 2*aa*cc*m + 3*ddp*y + aa*ddp*y + cc*ddp*y)
    - (y^3*(-(aa*cc) + ddp*y))/(aa*cc)) }

```

```
In[45]:= ClearAll[tau, recurrenceRaw, derivedP];
```

```
recurrenceRaw =
```

```
ode /. {g[y] => tau[m] / y^m, Derivative[j_][g][y] => tau[m + j] / y^(m + j)};
```

```
recurrenceRaw = Expand@Cancel@Together[y^m recurrenceRaw];
```

```
derivedP = {Factor[Coefficient[recurrenceRaw, tau[m]]],  
  Factor[Coefficient[recurrenceRaw, tau[m + 1]]],  
  Factor[Coefficient[recurrenceRaw, tau[m + 2]]],  
  Factor[Coefficient[recurrenceRaw, tau[m + 3]]]};
```

```
derivedP // InputForm
```

```
Out[49]//InputForm=
```

```
{(ddp*(d - m)*(aa + m)*(cc + m)*y)/(aa*cc), (aa*bb*cc*ddp + aa*bb*cc*m + aa*cc*ddp*  
cc*ddp*y - aa*cc*ddp*y + d*ddp*y + aa*d*ddp*y + cc*d*ddp*y - 3*ddp*m*y - 2*aa*ddp*  
3*ddp*m^2*y)/(aa*cc), (aa*cc + aa*bb*cc + aa*cc*ddp + 2*aa*cc*m - 3*ddp*y - aa*d*  
(aa*cc), (aa*cc - ddp*y)/(aa*cc)}
```

```
In[50]:= ClearAll[a0, b0, c0, epsilon0, beta0, gamma0, expectedP, recurrenceComparison];
```

```
a0 = 1 - y / aa;
```

```
b0[j_] := bb + j - y + (d - 1 - 2 j) y / aa;
```

```
c0[j_] := (d - j) (1 + j / aa) y;
```

```
epsilon0 = (cc - ddp) / cc;
```

```
beta0[j_] := aa (2 j + 1 - d) - d (2 j + 1) + 3 j^2 + 3 j + 1;
```

```
gamma0[j_] := j (j - d) (j + aa);
```

```
expectedP = {(ddp + m) c0[m] + epsilon0 y / aa gamma0[m],  
  c0[m + 1] + (ddp + m) b0[m] + epsilon0 y / aa beta0[m],  
  b0[m + 1] + (ddp + m) a0 + epsilon0 y / aa (aa - d + 3 + 3 m), a0 + epsilon0 y / aa};
```

```
recurrenceComparison = Table[Cancel@Together[derivedP[[j]] - expectedP[[j]], {j, 1, 4}];
```

```
recurrenceComparison // InputForm
```

```
Out[59]//InputForm=
```

```
{0, 0, 0, 0}
```

```
In[60]:= ClearAll[hyperPoly, odeResidual, exactTupleCheck, testTuples, polynomialTests];
```

```
hyperPoly[dv_, av_, bv_, cv_, ev_] :=
  Sum[Pochhammer[-dv, j] * Pochhammer[av, j] * Pochhammer[cv, j] *
    (ev y / (av cv))^j / (Pochhammer[bv, j] * Pochhammer[ev, j] * j!), {j, 0, dv}];

odeResidual[dv_, av_, bv_, cv_, ev_, mm_] :=
  Module[{poly, gm, lambdaValue, thetaValue, shiftValue, result},
    poly = hyperPoly[dv, av, bv, cv, ev];
    gm = D[poly, {y, mm}];
    lambdaValue = ev / (av cv);
    thetaValue[expr_] := Expand[y D[expr, y]];
    shiftValue[expr_, constant_] := Expand[thetaValue[expr] + constant expr];
    result = thetaValue[shiftValue[shiftValue[gm, ev + mm - 1], bv + mm - 1]] -
      lambdaValue y * shiftValue[shiftValue[shiftValue[gm, cv + mm], av + mm], mm - dv];
    Factor@Cancel@Together[result]];
```

```
testTuples = {{5, 12, 9, 7, 3}, {8, 25, 13, 9, 4}, {11, 40, 19, 12, 5}};
```

```
polynomialTests =
  Table[Module[{dv, av, bv, cv, ev, residuals}, {dv, av, bv, cv, ev} = tuple;
    residuals = Table[odeResidual[dv, av, bv, cv, ev, mm], {mm, 0, dv}];
    <|"Parameters" -> tuple, "OrdersTested" -> Range[0, dv],
      "Pass" -> And@@ (# === 0 & /@ residuals) |>], {tuple, testTuples}];
```

```
polynomialTests // InputForm
```

```
Out[65]//InputForm=
```

```
{<|"Parameters" -> {5, 12, 9, 7, 3}, "OrdersTested" -> {0, 1, 2, 3, 4, 5}, "Pass" ->
  <|"Parameters" -> {8, 25, 13, 9, 4}, "OrdersTested" -> {0, 1, 2, 3, 4, 5, 6, 7, 8};
  <|"Parameters" -> {11, 40, 19, 12, 5}, "OrdersTested" -> {0, 1, 2, 3, 4, 5, 6, 7, 8}}
```

```

In[66]:= ClearAll[nn, lf, saddleEquation, implicitEquation, rawRule,
          derivedLPrime, qDerived, m1ImplicitCheck, m1ImplicitResult];

saddleEquation = nn == lf[nn] (Pi Exp[lf[nn]] + 3 / 4);

implicitEquation = D[saddleEquation, nn];

rawRule = First@First@Solve[implicitEquation, lf'[nn]];

derivedLPrime = Factor@Together[(lf'[nn] /. rawRule) /.
    Pi Exp[lf[nn]] -> nn / lf[nn] - 3 / 4];

qDerived = (1 + lf[nn]) nn - (3 / 4) lf[nn]^2;

m1ImplicitCheck = Cancel@Together[derivedLPrime - lf[nn] / qDerived];

m1ImplicitResult = <|"RawImplicitRule" -> rawRule,
    "ReducedLPrime" -> derivedLPrime, "Q" -> qDerived, "ExactCheck" -> m1ImplicitCheck|>;

m1ImplicitResult // InputForm

Out[74]//InputForm=
<|"RawImplicitRule" -> Derivative[1][lf][nn] -> 4/(3 + 4*E^lf[nn]*Pi + 4*E^lf[nn]*P-
    "ReducedLPrime" -> (-4*lf[nn])/(-4*nn - 4*nn*lf[nn] + 3*lf[nn]^2), "Q" -> (-3*lf[nr
    "ExactCheck" -> 0|>

```

```
In[75]:= ClearAll[ell, q0, ellPrime0, g0, totalD, gDerivative, derivativeSetupCheck];
```

```
q0 = (1 + ell) nn - (3 / 4) ell^2;
```

```
ellPrime0 = ell / q0;
```

```
g0 = (nn + 1) Log[ell] + ell / 4 - nn / ell - (1 / 2) Log[q0];
```

```
totalD[expr_] := Cancel@Together[D[expr, nn] + ellPrime0 D[expr, ell]];
```

```
gDerivative[0] = g0;
```

```
gDerivative[order_Integer?Positive] :=  
  gDerivative[order] = totalD[gDerivative[order - 1]];
```

```
gDerivative[1];
```

```
gDerivative[2];
```

```
derivativeSetupCheck = <|"Order1LeafCount" → LeafCount[gDerivative[1]],  
  "Order2LeafCount" → LeafCount[gDerivative[2]],  
  "Order2ContainsLog" → Not[FreeQ[gDerivative[2], _Log]],  
  "Order2DenominatorNonzeroExpression" → Denominator[gDerivative[2]] != 0|>;
```

```
derivativeSetupCheck // InputForm
```

```
Out[85]//InputForm=
```

```
<|"Order1LeafCount" -> 88, "Order2LeafCount" -> 114, "Order2ContainsLog" -> False,  
  "Order2DenominatorNonzeroExpression" -> True|>
```

```

In[86]:= ClearAll[rr, ss, reducedDerivative,
           preChainLowOrders, postChainLowOrders, lowOrderResult];

gDerivative[3];
gDerivative[4];

reducedDerivative[order_Integer?Positive] :=
  reducedDerivative[order] = Cancel@Together[
    (nn^(order - 1) * ell * gDerivative[order]) /. {nn → 1 / (rr ss), ell → 1 / rr}];

preChainLowOrders = Table[Cancel@Together[reducedDerivative[order] /.
  {rr → 0, ss → 0}], {order, 2, 4}];

(*The substitution N=
  2x-2 contributes a net factor 2 to each normalized leading coefficient.*)
postChainLowOrders = 2 preChainLowOrders;

lowOrderResult = <|"Order3LeafCount" → LeafCount[gDerivative[3]],
  "Order4LeafCount" → LeafCount[gDerivative[4]], "PreChainConstants2To4" →
  preChainLowOrders, "PostChainConstants2To4" → postChainLowOrders|>;

lowOrderResult // InputForm
Out[93]//InputForm=
<|"Order3LeafCount" -> 210, "Order4LeafCount" -> 325, "PreChainConstants2To4" -> {1,
  "PostChainConstants2To4" -> {2, -2, 4}}|>

```



```

In[94]:= ClearAll[reduced5, fifthSigmaZero,
           fifthPreChainConstant, fifthPostChainConstant, m1Order5Result];

gDerivative[5];

reduced5 = reducedDerivative[5];

fifthSigmaZero = Factor@Cancel@Together[reduced5 /. ss → 0];

fifthPreChainConstant = Cancel@Together[reduced5 /. {rr → 0, ss → 0}];

fifthPostChainConstant = 2 fifthPreChainConstant;

m1Order5Result = <|"Order5LeafCount" → LeafCount[gDerivative[5]],
                  "Reduced5LeafCount" → LeafCount[reduced5], "SigmaZeroReduction" → fifthSigmaZero,
                  "PreChainConstant" → fifthPreChainConstant,
                  "PostChainConstant" → fifthPostChainConstant|>;

m1Order5Result // InputForm
Out[101]//InputForm=
<|"Order5LeafCount" -> 464, "Reduced5LeafCount" -> 413, "SigmaZeroReduction" ->
  -((6 + 47*rr + 146*rr^2 + 210*rr^3 + 126*rr^4 + 42*rr^5 + 6*rr^6)/(1 + rr)^7), "Pr
  "PostChainConstant" -> -12|>

```

In[102]:=

```

ClearAll[reduced6, sixthDenominator, sixthDenominatorRatio,
  sixthPreChainConstant, sixthPostChainConstant, m1Order6Result];

gDerivative[6];

reduced6 = reducedDerivative[6];

sixthDenominator = Factor[Denominator[reduced6]];

sixthDenominatorRatio =
  Factor@Cancel@Together[sixthDenominator / (4 + 4 rr - 3 ss) ^ 12];

sixthPreChainConstant = Cancel@Together[reduced6 /. {rr → 0, ss → 0}];

sixthPostChainConstant = 2 sixthPreChainConstant;

m1Order6Result = <|"Order6LeafCount" → LeafCount[gDerivative[6]],
  "Reduced6LeafCount" → LeafCount[reduced6],
  "Reduced6Denominator" → sixthDenominator,
  "DenominatorRatioToExpected" → sixthDenominatorRatio, "PreChainConstant" →
    sixthPreChainConstant, "PostChainConstant" → sixthPostChainConstant|>;

m1Order6Result // InputForm

```

Out[110]//InputForm=

```

<|"Order6LeafCount" -> 627, "Reduced6LeafCount" -> 566, "Reduced6Denominator" -> (4
  "DenominatorRatioToExpected" -> 1, "PreChainConstant" -> 24, "PostChainConstant" ->

```

```

In[111]:=
ClearAll[numerator6, coefficientRules6,
  numeratorTermCount, numeratorTotalDegree, m4StructureResult];

numerator6 = Expand[Numerator[reduced6]];

coefficientRules6 = CoefficientRules[numerator6, {rr, ss}];

numeratorTermCount = Length[coefficientRules6];

numeratorTotalDegree = Max[Total[First[#]] & /@ coefficientRules6];

m4StructureResult = <|"PolynomialQ" → PolynomialQ[numerator6, {rr, ss}],
  "ContainsMachineReal" → Not[FreeQ[numerator6, _Real]],
  "TermCount" → numeratorTermCount, "TotalDegree" → numeratorTotalDegree|>;

m4StructureResult // InputForm
Out[117]//InputForm=
<|"PolynomialQ" -> True, "ContainsMachineReal" -> False, "TermCount" -> 82, "TotalDegree" -> 82|>;

In[118]:=
ClearAll[boxRadius, numeratorMajorant,
  denominatorLowerBound, h6Majorant, m4MajorantResult];

boxRadius = 7 / 50;

numeratorMajorant =
  Total[Abs[Last[#]] * boxRadius ^ Total[First[#]] & /@ coefficientRules6];

denominatorLowerBound = (151 / 50) ^ 12;

h6Majorant = Cancel@Together[numeratorMajorant / denominatorLowerBound];

m4MajorantResult = <|"BoxRadius" → boxRadius,
  "DenominatorLowerBound" → denominatorLowerBound,
  "H6Majorant" → h6Majorant, "StrictlyLessThan10000" → h6Majorant < 10 000|>;

m4MajorantResult // InputForm
Out[124]//InputForm=
<|"BoxRadius" -> 7/50, "DenominatorLowerBound" -> 140515219945627518837736801/244146,
  "H6Majorant" -> 6422139805764931584036533551104/702576099728137594188684005, "StrictlyLessThan10000" -> True|>;

```

In[125]:=

```

ClearAll[acceptanceRecord, mathematicalResults, exactArithmeticCheck,
  resultLedger, ledgerPath, ledgerHash, ledgerFreezeResult];

acceptanceRecord = <|"M1" → "MATCH", "M2" → "MATCH", "M3" → "MATCH", "M4" → "MATCH"|>;

mathematicalResults = <|
  "M1" → <|"ImplicitDerivative" → m1ImplicitResult, "ReducedDerivatives2Through6" →
    Association@Table[ToString[order] → reducedDerivative[order], {order, 2, 6}],
  "Order5Check" → m1Order5Result, "Order6Check" → m1Order6Result|>,
  "M2" → <|"CoefficientRatio" → coefficientRatio, "CrossMultipliedCheck" →
    crossMultipliedCheck, "DerivedRecurrenceCoefficients" → derivedP,
  "RecurrenceComparisons" → recurrenceComparison,
  "ExactPolynomialTests" → polynomialTests|>, "M3" → m3Results,
  "M4" → <|"Structure" → m4StructureResult, "NumeratorMajorant" → numeratorMajorant,
  "MajorantResult" → m4MajorantResult|>|>;

exactArithmeticCheck = FreeQ[mathematicalResults, _Real];

resultLedger = <|"Metadata" → metadata,
  "MethodDisclosure" → "Mathematica cells supplied interactively by Codex,
    entered and executed by the user; no repository JSON, Python, SymPy,
    Lean-generated expression, or frozen numerator was imported.",
  "Acceptance" → acceptanceRecord, "ExactResultsContainNoMachineReals" →
    exactArithmeticCheck, "Results" → mathematicalResults|>;

ledgerPath =
  FileNameJoin[{NotebookDirectory[], "C48_Mathematica_Result_Ledger.txt"}];

Export[ledgerPath, ToString[resultLedger, InputForm], "Text"];

ledgerHash = IntegerString[FileHash[ledgerPath, "SHA256"], 16, 64];

ledgerFreezeResult = <|"LedgerPath" → ledgerPath,
  "LedgerBytes" → FileByteCount[ledgerPath], "ExactResultsContainNoMachineReals" →
    exactArithmeticCheck, "LedgerSHA256" → ledgerHash|>;

ledgerFreezeResult // InputForm

```

Out[134]//InputForm=

```

<|"LedgerPath" -> "/Users/jsavva/Desktop/Jensen_Mathemtica_Cleanroom/C48_Mathematica
  "LedgerBytes" -> 7105, "ExactResultsContainNoMachineReals" -> True,
  "LedgerSHA256" -> "1faea5fcb35b504b5d0aad9391999a99e530373dce36addb496eabb133fb04cc

```

In[135]:=

```

ClearAll[notebookPath, pdfPath, hashPath, notebookHash,
  pdfHash, finalLedgerHash, hashRows, artifactFreezeResult];

notebookPath = NotebookFileName[];

pdfPath = FileNameJoin[{NotebookDirectory[], "C48_Mathematica_CleanRoom.pdf"}];

hashPath = FileNameJoin[{NotebookDirectory[], "SHA256SUMS.txt"}];

(*Save the complete evaluated notebook currently on screen.*)
NotebookSave[EvaluationNotebook[], notebookPath];

(*Render the saved calculation record as PDF.*)
NotebookPrint[EvaluationNotebook[], pdfPath];

notebookHash = IntegerString[FileHash[notebookPath, "SHA256"], 16, 64];

pdfHash = IntegerString[FileHash[pdfPath, "SHA256"], 16, 64];

finalLedgerHash = IntegerString[FileHash[ledgerPath, "SHA256"], 16, 64];

hashRows = {notebookHash <> "  " <> FileNameTake[notebookPath], pdfHash <> "  " <>
  FileNameTake[pdfPath], finalLedgerHash <> "  " <> FileNameTake[ledgerPath]};

Export[hashPath, StringRiffle[hashRows, "\n"] <> "\n", "Text"];

artifactFreezeResult = <|"NotebookPath" → notebookPath,
  "NotebookSHA256" → notebookHash, "PDFPath" → pdfPath,
  "PDFSHA256" → pdfHash, "LedgerPath" → ledgerPath,
  "LedgerSHA256" → finalLedgerHash, "ChecksumFile" → hashPath, "AllFilesExist" →
    And @@ FileExistsQ /@ {notebookPath, pdfPath, ledgerPath, hashPath}>];

artifactFreezeResult // InputForm

```